Templates de clases

Si recordamos alguna de las estructuras vista en C, como por ejemplo la pila; vemos que habíamos creado una estructura completa pero solo aplicada a enteros. De tal manera que si hubiésemos querido reutilizar esto para otros tipos de datos, era necesario desarrollar todo el código completo nuevamente. Aplicando templates como en las funciones, lo vamos a aplicar a la estructura y a la clase, de tal manera de hacer una pila genérica, valiendo también para colas, listas y cualquier otra estructura. Si hablamos de una lista estaríamos implementando en este caso algo similar a la estructura de List<Tipo_de_dato> que vemos en lenguajes de alto nivel como C# o Java.

Veamos la pila que teníamos:

```
typedef struct {
    int elementos[100];
    int tope;
} Pila;

void apilar(Pila *p, int valor);
int  desapilar(Pila *p);
```

Solo sirve para int. Si mañana necesitan una pila de double o de Alumno, hay que duplicar toda la estructura y todas sus funciones.Los datos están sueltos. Nada impide que alguien escriba p.tope = 50; desde afuera y rompa la invariante de la estructura.

Ahora resolvamos este problema aplicando POO y templates:

```
#define stacksize 5
template <typename T>
class pila
{
private:
    T elementos[stacksize];
    int tope;

public:
    void apilar(T );
    T desapilar();
};
```

Instanciando esto en un main podríamos hacer algo así:

```
Pila<int>    pilaEnteros;
Pila<double> pilaReales;
Pila<Alumno> pilaAlumnos;
```

Al declarar los métodos de la clase por fuera de la misma, debemos volver a declarar el template en cada nueva definición:

```
template <typename T>
T pila<T>::desapilar()
{
    T num=this->elementos[this->tope];
    this->tope--;
    return num;
}
```